

UNIVERSIDAD PRIVADA FRANZ TAMAYO

“UNIFRANZ”

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS

INGENIERÍA DE SISTEMAS



**"INFORME CONSOLIDADO: SISTEMA WEB DE ADMINISTRACIÓN CON MACHINE
LEARNING"**

Caso: Minimarket Doña Elena – El Alto, Bolivia

ESTUDIANTE:

FRANCISCO JOSE CALLE PARI

DOCENTE:

ING. JAVIER ELVIS CANQUI LLUSCO

EL ALTO - BOLIVIA

GESTIÓN 2026

Capítulo I: Marco Introductorio

1.1 Introducción

En el actual ecosistema comercial de las ciudades de La Paz y El Alto, el sector minorista y de minimarkets enfrenta un rezago tecnológico crítico. La dependencia de métodos tradicionales de registro, como cuadernos manuales o planillas básicas de Excel, genera un flujo administrativo propenso a errores, mermas económicas y una ceguera administrativa que impide el crecimiento sostenido. El presente proyecto aborda la modernización de la gestión operativa en el Minimarket Doña Elena mediante la implementación de un sistema web integral que no solo digitalice el punto de venta y los inventarios, sino que introduzca una capa predictiva impulsada por algoritmos de Machine Learning, transformando los datos históricos en decisiones estratégicas inteligentes.

1.2 Planteamiento del Problema

La gestión de inventarios y ventas en la Tienda Minimarket Doña Elena se realiza de forma empírica y manual. Esto genera dos escenarios perjudiciales: el capital dormido, causado por el sobrestock y posterior vencimiento de productos perecederos (lácteos, embutidos), y el quiebre de stock, consistente en el desabastecimiento de productos de alta rotación en días pico (ferias de jueves y domingo). La ausencia de datos estructurados imposibilita cualquier tipo de proyección comercial o inteligencia de negocios.

En base a este contexto, se plantea la siguiente Pregunta Esencial:

¿De qué manera se realiza la implementación de algoritmos de Machine Learning para la predicción de demanda de productos y la optimización de inventarios en el Minimarket Doña Elena de la ciudad de El Alto, para mejorar su eficiencia operativa y competitividad frente a la informalidad?

1.3 Objetivos

Objetivo General:

Desarrollar un sistema web de administración integrado con modelos de Inteligencia Artificial para la gestión inteligente de inventarios y ventas en el Minimarket Doña Elena

Objetivos Específicos:

- Desarrollar el módulo de inventarios base para registrar productos, categorías y proveedores centralizando el almacenamiento de datos transaccionales
- Implementar el módulo de Punto de Venta (POS) para procesar y registrar salidas de productos en tiempo real, garantizando la trazabilidad del stock
- Desarrollar un motor predictivo basado en Machine Learning utilizando Python para la generación de alertas tempranas de vencimiento y reabastecimiento inteligente
- Diseñar un módulo de reportes gerencial para visualizar gráficos y KPIs operativos derivados del análisis del histórico de ventas

Capítulo II: Marco Teórico

2.1 Definiciones Enfocadas al Sistema

De acuerdo con fundamentos de la ingeniería de software moderna propuestos por autores como Pressman y Sommerville, el proyecto se clasifica como un Sistema Transaccional con Arquitectura Cliente-Servidor

- Sistema POS (Point of Sale): Arquitectura híbrida (hardware/software) encargada del registro inmediato de transacciones comerciales, cálculo de totales y deducción concurrente de inventarios
- Machine Learning (Aprendizaje Supervisado): Subcampo de la Inteligencia Artificial utilizado en este sistema mediante algoritmos de regresión lineal y series de tiempo, donde el software identifica patrones históricos de ventas para proyectar demandas futuras de forma automatizada y reducir el error humano

2.2 Definiciones Enfocadas a la Metodología

- Scrum: Marco de trabajo ágil utilizado para el ciclo de desarrollo de software, estructurado en ciclos cortos (Sprints) que aseguran la entrega continua de módulos funcionales priorizados por el cliente
- CRISP-DM: Metodología estándar para el ciclo de vida de la ciencia de datos (comprensión del negocio, de los datos, preparación, modelado, evaluación y despliegue)
- UML (Lenguaje Unificado de Modelado): Estándar de ingeniería que permite visualizar el comportamiento del sistema mediante Casos de Uso (para la interacción de los actores) y modelado de la arquitectura lógica de datos mediante el Diagrama Entidad-Relación (DER)

2.3 Definiciones Enfocadas al Problema

- Quiebre de Stock: Nivel nulo de existencias de un artículo específico provocado por deficiencias de previsión empírica, resultando en ventas perdidas frente a la competencia
- Capital Dormido: Dinero de la empresa inmovilizado en mercancía física de baja rotación por fallos en las decisiones de compra basadas en intuición
- Control Empírico (Visual): Práctica deficiente de administración minorista en la cual la reposición de stock obedece únicamente a la revisión ocular de las repisas, eludiendo cualquier métrica de consumo histórico

Capítulo III: Marco de Desarrollo

3.1 Ingeniería de Requisitos

Especificación de Requerimientos Funcionales (RF)

- RF 1: Registro Centralizado de Productos
 - Hallazgo: La propietaria pierde el control físico de la mercancía ingresada porque se registra en cuadernos propensos a tachaduras o pérdida
 - Requisito: El administrador registrará la información base de los productos mediante el llenado de un formulario web validado para evitar códigos duplicados en el inventario
- RF 2: Actualización de Inventario vía Punto de Venta (POS)
 - Hallazgo: La falta de registro de salidas instantáneas genera un desfase crítico entre lo anotado y lo que realmente existe en tienda.
 - Requisito: El cajero actualizará los niveles de stock escaneando o buscando productos en el módulo POS para descontar las cantidades automáticamente tras cada venta
- RF 3: Alertas por Vencimiento de Lote
 - Hallazgo: Se genera capital inmovilizado al caducar lácteos y embutidos en el fondo del estante sin que el comerciante lo note a tiempo.
 - Requisito: El sistema generará una alerta visual en el dashboard principal cuando la fecha de caducidad de un producto sea menor o igual a 7 días
- RF 4: Notificación por Stock Mínimo
 - Hallazgo: Durante días de feria (jueves y domingos) la tienda se queda repentinamente sin artículos de primera necesidad (pan, bebidas)
 - Requisito: El sistema notificará de manera automática en la interfaz gerencial si el nivel de existencias de un producto alcanza su nivel mínimo previamente configurado

- RF 5: Exportación de Reportes Gerenciales
 - Hallazgo: El análisis de rentabilidad es nulo porque sumar los apuntes del cuaderno tomaría días de trabajo manual exclusivo
 - Requisito: El supervisor exportará reportes estructurados de inventario y ventas utilizando botones de acción en formatos estándar (PDF/Excel)

Especificación de Requerimientos No Funcionales (RNF)

- RNF 1: Rendimiento en Transacciones
 - Hallazgo: El ritmo acelerado del despacho minorista no tolera esperas prolongadas en la atención al cliente en mostrador
 - Requisito: El servidor procesará las búsquedas de productos y finalizaciones de venta garantizando un tiempo de respuesta máximo de 2 segundos
- RNF 2: Respaldo y Disponibilidad (Seguridad)
 - Hallazgo: Toda la gestión depende de hardware convencional, que en caso de fallo eléctrico compromete la continuidad del negocio.
 - Requisito: El motor de base de datos ejecutará un script de respaldo automático de manera recurrente a las 23:00 hrs para prevenir la pérdida de registros.

Priorización: Matriz MoSCoW

Categoría	Requisito	Tipo	Justificación MVP
M (Must Have)	Actualización vía POS	RF2	Crítico para que el stock represente la realidad del negocio y permita operar comercialmente.
M (Must Have)	Registro Centralizado	RF1	Sin base de datos de productos no hay sistema que usar.
M (Must Have)	Rendimiento < 2s	RNF1	Imprescindible para evitar cuellos de botella frente a los clientes.
S (Should Have)	Notificación Stock Mínimo	RF4	Esencial para el control inteligente, aunque la tienda puede sobrevivir los primeros días operando visualmente.

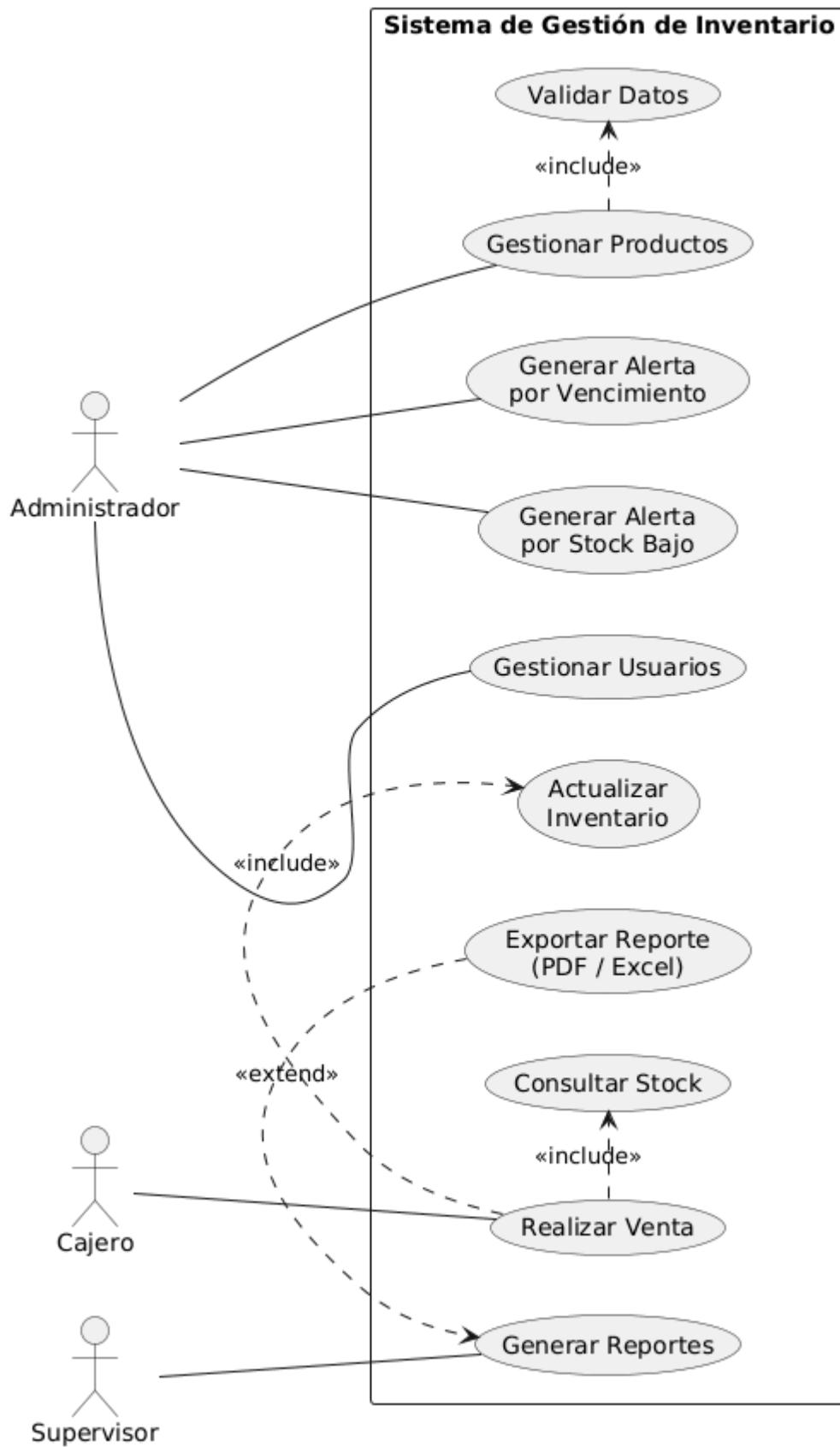
Categoría	Requisito	Tipo	Justificación MVP
S (Should Have)	Alerta Vencimiento	RF3	Altamente prioritario para reducir el 'capital dormido'.
C (Could Have)	Respaldo Automático	RNF2	Buena práctica técnica, delegable a ejecución manual en la primera semana operativa.
W (Won't Have)	Exportación de Reportes	RF5	Se posterga para sprints posteriores, ya que la prioridad inicial es recabar la data a través del POS.

Matriz de Trazabilidad (RTM)

Objetivo Específico	Requisito Funcional	Caso de Uso UML	Entidad Principal (DER)
1. Desarrollar módulo de inventarios	RF 1: Registro Centralizado	CU-01: Gestionar productos	Producto / Categoría
2. Implementar módulo POS	RF 2: Actualización vía POS	CU-02: Realizar venta	Venta / Detalle_Venta
3. Desarrollar motor predictivo	RF 3: Alertas Vencimiento	CU-03: Alerta de Vencimiento	Producto / Lote
3. Desarrollar motor predictivo	RF 4: Notificación Stock	CU-04: Alerta Stock Mínimo	Producto (campo: stock_minimo)
4. Diseñar reportería gerencial	RF 5: Exportación Reportes	CU-05: Generar Reportes	Venta / Inventario

3.2 Modelado de Comportamiento y Datos

Diagrama UML de Casos de Uso (Arquitectura Lógica)



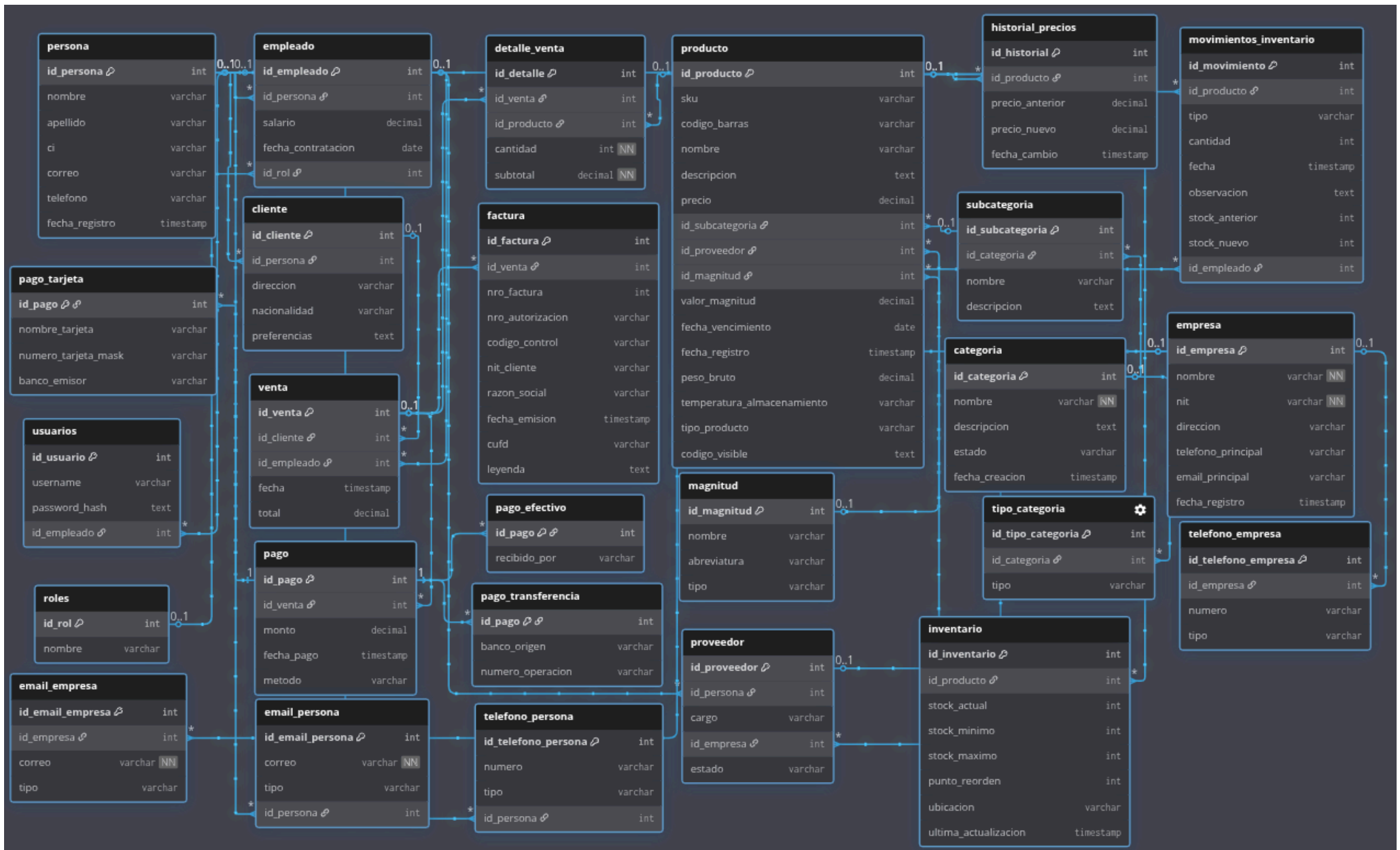
El flujo de Casos de Uso sigue una estructura modular.

- Actor Administrador: Acciona CU-01 Gestionar Productos (incluye validación de datos), CU-03 Generar alerta por vencimiento, CU-04 Generar alerta por stock bajo y CU-06 Gestionar Usuarios.
- Actor Cajero: Acciona CU-02 Realizar Venta (el cual incluye CU-09 Consultar Stock y Actualizar Inventario).
- Actor Supervisor: Acciona CU-05 Generar Reportes, el cual tiene como caso extendido la acción de Exportar Reporte (PDF/Excel).

Escenario de Éxito Estructurado (Módulo Crítico: POS - CU-02)

- El Cajero accede a la pantalla del Punto de Venta
- El Cajero ingresa el código de barras o nombre parcial del producto
- El sistema valida y muestra la información del producto y el stock disponible
- El Cajero establece la cantidad y agrega el ítem al carrito de compras virtual
- El Cajero presiona el botón de facturar e ingresa el monto recibido
- El sistema deduce el stock inmediatamente en la base de datos y registra el evento de salida en la tabla Venta
- El sistema arroja el comprobante visual y el mensaje de confirmación Venta exitosa.

Diagrama Entidad-Relación (DER) Lógico



El núcleo transaccional gira en torno a las siguientes relaciones principales:

- Una Categoría (1) agrupa a Muchos (N) Productos.
- Un Producto (1) tiene asociado un Lote con atributos de vencimiento y stock específico.
- Una Venta (1) contiene Muchos (N) Detalles_Venta.
- Un Detalle_Venta (N) hace referencia a Un (1) único Producto.
- Un Usuario (1) registra Muchas (N) Ventas.

Diccionario de Datos (Muestra de Entidades Críticas)

Tabla: Producto

Campo	Tipo de Dato	Llave	Condición / Restricción
id_producto	Integer	PK	Auto-incremental, NOT NULL
sku	Varchar(255)	UK	Opcional, único
codigo_barras	Varchar(255)	UK	Opcional, único
nombre	Varchar(255)	-	NOT NULL
descripcion	Text	-	Opcional
precio	Decimal(10,2)	-	NOT NULL, mayor que 0
id_subcategoria	Integer	FK	Referencia a Subcategoria
id_proveedor	Integer	FK	Referencia a Proveedor
id_magnitud	Integer	FK	Referencia a Magnitud
valor_magnitud	Decimal(10,2)	-	Opcional
fecha_vencimiento	Date	-	Opcional
fecha_registro	Timestamp	-	Fecha de creación
peso_bruto	Decimal(10,2)	-	Opcional
temperatura_almacenamiento	Varchar(100)	-	Opcional
tipo_producto	Varchar(50)	-	Opcional
codigo_visible	Text	-	Opcional

Tabla: Venta

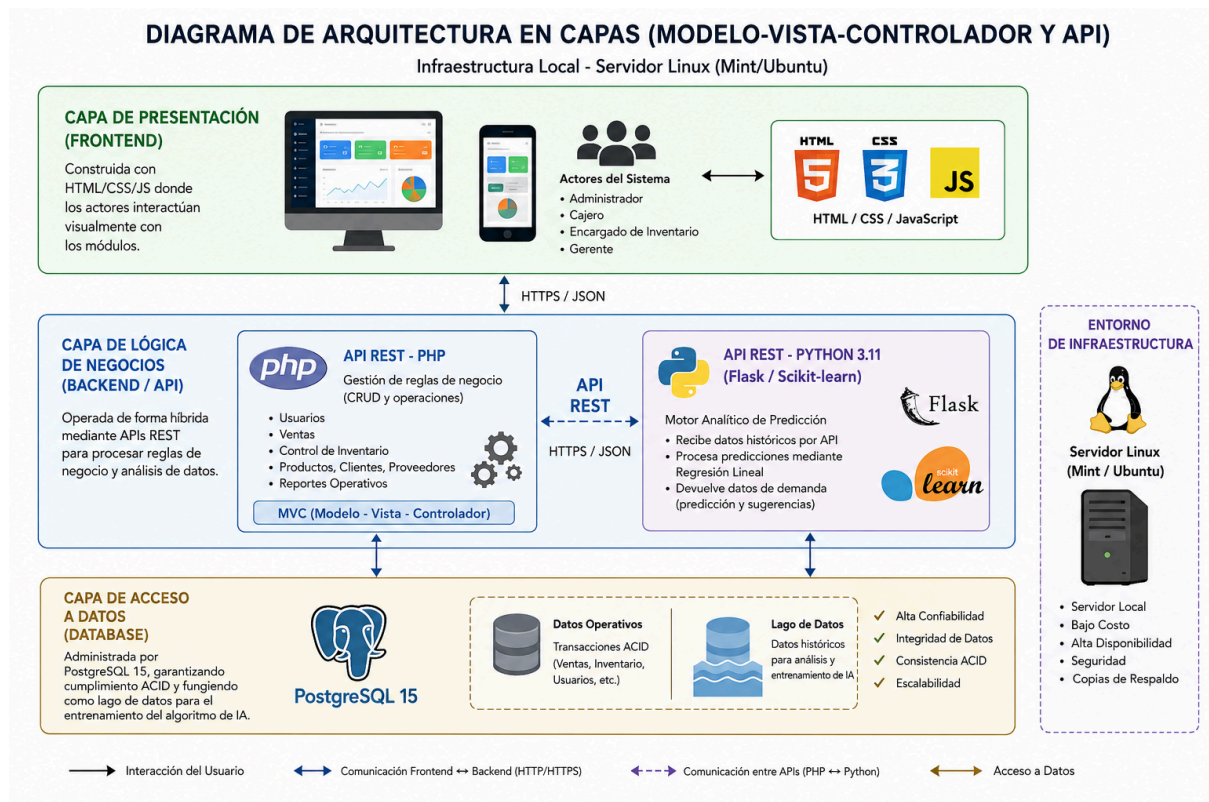
Campo	Tipo de Dato	Llave	Condición / Restricción
id_venta	Integer	PK	Auto-incremental, NOT NULL
id_cliente	Integer	FK	Referencia a Cliente
id_empleado	Integer	FK	Referencia a Empleado
fecha	Timestamp	-	Current_Timestamp
total	Decimal(10,2)	-	NOT NULL, mayor que 0

Tabla: Inventario

Campo	Tipo de Dato	Llave	Condición / Restricción
id_inventario	Integer	PK	Auto-incremental, NOT NULL
id_producto	Integer	FK	NOT NULL (Ref: Producto)
stock_actual	Integer	-	NOT NULL, mayor o igual a 0
stock_minimo	Integer	-	Opcional
stock_maximo	Integer	-	Opcional
punto_reorden	Integer	-	Opcional
ubicacion	Varchar(255)	-	Opcional
ultima_actualizacion	Timestamp	-	Fecha de actualización

3.3 Arquitectura de Software y Diseño UX

Diagrama de Arquitectura en Capas



El sistema se implementa en una infraestructura local usando un servidor Linux (Mint/Ubuntu) para abaratar costos, aplicando el patrón Modelo-Vista-Controlador (MVC) y APIs REST:

- Capa de Presentación (Frontend): Construida con HTML/CSS/JS donde los actores interactúan visualmente con los módulos
- Capa de Lógica de Negocios (Backend/API): Operada de forma híbrida.
 - PHP: Gestiona las reglas de negocio CRUD (Usuarios, Ventas, Control de Inventario)
 - Python 3.11 (Flask / Scikit-learn): Motor analítico encapsulado que recibe datos históricos, procesa predicciones mediante Regresión Lineal y devuelve datos de demanda
- Capa de Acceso a Datos (Database): Administrada por PostgreSQL 15, garantizando cumplimiento ACID para las miles de transacciones operativas y fungiendo como lago de datos para el entrenamiento del algoritmo de IA

Prototipo de Interfaz (Wireframes Mapping)



Wireframe del Formulario Registro de Productos:

<https://www.figma.com/design/yyg0ClvhqBNxlw62xHnvRk/Sin-t%C3%ADtulo?t=AcMGfF1rrih2XYvG-1>